

Variational Monte Carlo with Neural Quantum States

Gaussian-binary restricted Boltzmann machines for harmonic quantum dots

Marius Torsheim

Department of Physics, University of Oslo

Spring 2026

Abstract

Neural quantum states provide compact variational representations of high-dimensional wave functions. In this work I implemented a variational Monte Carlo solver where the trial wave function is a Gaussian-binary restricted Boltzmann machine. The analytical local energy, parameter-gradient estimators, and quantum force were derived in closed form and used in both brute-force Metropolis and Langevin importance sampling. The implementation was validated on non-interacting harmonic oscillator systems, for which the exact energy is $E = PD/2$ at $\omega = 1$. For one particle in one dimension, brute-force Metropolis gave $E = 0.499984 \pm 2.1 \cdot 10^{-6}$, while importance sampling gave $E = 0.499988 \pm 1.7 \cdot 10^{-6}$. For two particles in two dimensions without interaction, the corresponding estimates were $E = 1.999999 \pm 3.0 \cdot 10^{-6}$ and $E = 1.999974 \pm 3.9 \cdot 10^{-6}$. The interacting two-particle two-dimensional quantum dot was treated as the symmetric spatial ground state, corresponding to a spin-singlet state for electrons. The best completed interacting run used importance sampling, $N_h = 16$, learning rate 0.005, and gave $E = 3.0826 \pm 0.0082$, with conservative blocking error 0.0243. A larger-hidden-layer check with $N_h = 32$ gave $E = 3.0965 \pm 0.0069$ and did not improve the energy. The deviation from the exact value $E = 3.0$ is larger than the statistical uncertainty, indicating that the remaining error is dominated by optimization and ansatz limitations rather than sampling noise.

Repository: https://github.com/Torsheim/FYS4411_Project_2

I Introduction

Quantum many-body problems are difficult because the wave function depends on many coordinates and often contains non-trivial correlations. Even for few-body systems, a useful numerical method should balance physical accuracy, computational cost, and flexibility. Variational Monte Carlo (VMC) is well suited for this because it converts the ground-state problem into the optimization of a trial wave function. The variational principle guarantees that the expectation value of the Hamiltonian is an upper bound to the ground-state energy, so the quality of the trial state can be judged directly from the energy.

In this project the trial wave function is represented by a restricted Boltzmann machine (RBM). This follows the neural-network quantum state idea introduced by Carleo and Troyer [1]. The RBM is used as an explicit analytical function of the particle coordinates. Its visible nodes represent the continuous particle coordinates, while the hidden layer introduces additional nonlinear correlations. Because the coordinates are continuous, I use a Gaussian-binary RBM, with Gaussian visible nodes and binary hidden nodes.

The work has three main goals. First, the analytical expressions for the local energy, parameter derivatives, and quantum force must be derived and implemented. Second, the implementation must be validated on non-interacting harmonic oscillator systems with known exact energies. Third, the Coulomb interaction must be included for two particles in two dimensions, where an exact benchmark is available: for $\omega = 1$, the interacting ground-state energy is $E = 3.0$ a.u. [2]. The optional replacement of

the RBM by a feed-forward neural network was not attempted; instead, the focus was on obtaining a clear and tested RBM/VMC implementation.

The report is organized as a scientific report rather than as separate answers to the assignment items. Section II describes the physical systems, the RBM wave function, the analytical derivatives, and the VMC estimators. Section III explains the numerical algorithms and implementation. Section IV presents validation tests, statistical analysis, and interacting quantum-dot results. Section V summarizes the main conclusions and possible improvements. The detailed analytical derivation is included in Appendix A, while code organization and reproducibility information are summarized in Appendix B.

II Theory

A Physical systems

The Hamiltonian studied in this project is

$$\hat{H} = \sum_{p=1}^P \left(-\frac{1}{2} \nabla_p^2 + \frac{1}{2} \omega^2 r_p^2 \right) + \sum_{p < q} \frac{1}{r_{pq}}, \quad (1)$$

where P is the number of particles, D is the spatial dimension, and $r_{pq} = |\mathbf{r}_p - \mathbf{r}_q|$. Atomic units are used. The interaction term is omitted in the first numerical tests. For $\omega = 1$ and no interaction, the exact ground-state energy is

$$E_0^{(0)} = \frac{1}{2} PD. \quad (2)$$

This gives $E_0 = 0.5$ for one particle in one dimension and $E_0 = 2.0$ for two particles in two dimensions.

The main interacting benchmark is the two-particle, two-dimensional harmonic quantum dot, with

$$\mathbf{X} = (x_1, y_1, x_2, y_2), \quad (3)$$

and

$$V(\mathbf{X}) = \frac{1}{2}\omega^2(x_1^2 + y_1^2 + x_2^2 + y_2^2) + \frac{1}{\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}}. \quad (4)$$

For $\omega = 1$, the exact interacting value is $E_0 = 3.0$ a.u. [2]. This value is used as the central benchmark for the interacting calculation. Throughout the report, energies are reported in atomic units (a.u.). The oscillator frequency is fixed to $\omega = 1$ in atomic units. Monte Carlo iteration numbers, acceptance rates, numbers of hidden units, and learning rates are dimensionless. The Langevin time step Δt is the dimensionless algorithmic time step used in the importance-sampling proposal.

B Spin and exchange symmetry

The Hamiltonian in Eq. (1) is spin independent, but the interpretation of the two-particle state depends on whether the particles are treated as bosons or electrons. The RBM ansatz used here is a positive coordinate-space wave function and represents the symmetric spatial ground-state component. For two electrons in the lowest oscillator state, the corresponding total fermionic wave function must be antisymmetric under exchange. This is obtained by combining the symmetric spatial state with an antisymmetric spin singlet,

$$S = 0. \quad (5)$$

The present implementation therefore describes the spatial part of the singlet ground state. It does not include an explicit Slater determinant and does not attempt to represent triplet states or general antisymmetric many-fermion wave functions. For bosons, the same symmetric spatial ansatz can be interpreted directly as a bosonic ground-state trial function.

C Variational Monte Carlo

For a normalized trial wave function $\Psi(\mathbf{X}; \alpha)$, the variational energy is

$$E[\alpha] = \frac{\int \Psi^*(\mathbf{X}; \alpha) \hat{H} \Psi(\mathbf{X}; \alpha) d\mathbf{X}}{\int |\Psi(\mathbf{X}; \alpha)|^2 d\mathbf{X}}. \quad (6)$$

In VMC this is written as an expectation value over the probability density

$$p(\mathbf{X}) = \frac{|\Psi(\mathbf{X})|^2}{\int |\Psi(\mathbf{X})|^2 d\mathbf{X}}. \quad (7)$$

The sampled quantity is the local energy

$$E_L(\mathbf{X}) = \frac{1}{\Psi(\mathbf{X})} \hat{H} \Psi(\mathbf{X}), \quad (8)$$

so that

$$E[\alpha] = \langle E_L \rangle_p. \quad (9)$$

The energy is minimized by changing the variational parameters α of the RBM.

D Gaussian-binary RBM wave function

The visible vector $\mathbf{X} \in \mathbb{R}^M$ contains all coordinates, with

$$M = PD. \quad (10)$$

The Gaussian-binary RBM wave function used in the code is

$$\Psi(\mathbf{X}) = \frac{1}{Z} \exp \left[- \sum_{k=1}^M \frac{(X_k - a_k)^2}{2\sigma^2} \right] \prod_{j=1}^{N_h} (1 + \exp \theta_j), \quad (11)$$

with hidden pre-activations

$$\theta_j = b_j + \frac{1}{\sigma^2} \sum_{k=1}^M X_k w_{kj}. \quad (12)$$

Here $\mathbf{a}$ are visible biases, $\mathbf{b}$ are hidden biases, $\mathbf{W}$ is the weight matrix, σ is the visible Gaussian width, and N_h is the number of hidden nodes. The hidden activation probability is

$$q_j = \text{sigmoid}(\theta_j) = \frac{1}{1 + \exp(-\theta_j)}. \quad (13)$$

The logarithmic form of Eq. (11) is especially useful,

$$\ln \Psi(\mathbf{X}) = \text{const.} - \sum_{k=1}^M \frac{(X_k - a_k)^2}{2\sigma^2} + \sum_{j=1}^{N_h} \ln(1 + \exp \theta_j). \quad (14)$$

The normalization constant does not enter any Metropolis ratio or coordinate derivative, and it cancels in the covariance form of the energy gradient.

E Local energy

The coordinate derivatives are most compactly written as logarithmic derivatives. Define

$$g_k = \frac{\partial \ln \Psi}{\partial X_k} = -\frac{X_k - a_k}{\sigma^2} + \frac{1}{\sigma^2} \sum_{j=1}^{N_h} w_{kj} q_j, \quad (15)$$

and

$$h_k = \frac{\partial^2 \ln \Psi}{\partial X_k^2} = -\frac{1}{\sigma^2} + \frac{1}{\sigma^4} \sum_{j=1}^{N_h} w_{kj}^2 q_j (1 - q_j). \quad (16)$$

Then

$$\frac{1}{\Psi} \frac{\partial^2 \Psi}{\partial X_k^2} = h_k + g_k^2. \quad (17)$$

The local energy is therefore

$$E_L(\mathbf{X}) = -\frac{1}{2} \sum_{k=1}^M (h_k + g_k^2) + \frac{1}{2} \omega^2 \sum_{k=1}^M X_k^2 + \sum_{p < q} \frac{1}{r_{pq}}. \quad (18)$$

For the non-interacting systems, the Coulomb term is removed. Appendix A gives the derivation in more detail.

F Parameter derivatives and optimization

The variational parameters are

$$\alpha = \{a_1, \dots, a_M, b_1, \dots, b_{N_h}, w_{11}, \dots, w_{MN_h}\}. \quad (19)$$

The logarithmic parameter derivatives are

$$O_{a_k}(\mathbf{X}) = \frac{1}{\Psi} \frac{\partial \Psi}{\partial a_k} = \frac{X_k - a_k}{\sigma^2}, \quad (20)$$

$$O_{b_j}(\mathbf{X}) = \frac{1}{\Psi} \frac{\partial \Psi}{\partial b_j} = q_j, \quad (21)$$

$$O_{w_{kj}}(\mathbf{X}) = \frac{1}{\Psi} \frac{\partial \Psi}{\partial w_{kj}} = \frac{X_k q_j}{\sigma^2}. \quad (22)$$

For any parameter α , the energy gradient is estimated as

$$\frac{\partial \langle E_L \rangle}{\partial \alpha} = 2 (\langle E_L O_\alpha \rangle - \langle E_L \rangle \langle O_\alpha \rangle). \quad (23)$$

The optimization in the production runs used ADAM [6]. In the notation of ordinary gradient descent, the update would be

$$\alpha^{(n+1)} = \alpha^{(n)} - \eta \frac{\partial \langle E_L \rangle}{\partial \alpha}, \quad (24)$$

where η is the learning rate.

G Importance sampling and quantum force

The importance-sampling version uses Langevin proposals. The quantum force is

$$F_k(\mathbf{X}) = 2 \frac{\partial \ln \Psi}{\partial X_k} = 2g_k. \quad (25)$$

A new configuration is proposed as

$$\mathbf{Y} = \mathbf{X} + D\mathbf{F}(\mathbf{X})\Delta t + \sqrt{\Delta t} \boldsymbol{\xi}, \quad (26)$$

where $D = 1/2$ and $\boldsymbol{\xi}$ is a normally distributed random vector. The Metropolis-Hastings acceptance probability includes the Green's-function ratio,

$$A(\mathbf{X} \rightarrow \mathbf{Y}) = \min \left(1, \frac{|\Psi(\mathbf{Y})|^2 G(\mathbf{Y} \rightarrow \mathbf{X})}{|\Psi(\mathbf{X})|^2 G(\mathbf{X} \rightarrow \mathbf{Y})} \right). \quad (27)$$

This is more expensive than brute-force Metropolis, but it can reduce random-walk behavior by guiding proposals toward regions where $|\Psi|^2$ is large.

H Blocking analysis

The local-energy samples are correlated because they are generated by a Markov chain. The naive standard error can therefore be too small. Blocking estimates the error by repeatedly grouping neighboring samples into blocks and computing the variance of the block averages [7]. In the present code, the sample file used for blocking contains the local-energy samples from the final optimization iteration. The first iteration of each run removes an explicit burn-in period, and later iterations continue from the previous final Markov-chain position. Thus the final sampled batch is close to equilibrium for the final optimized state, but it is not a completely independent fixed-parameter production run. This is a limitation of

the present analysis, and a stricter production protocol would freeze the optimized RBM parameters and generate a new, longer chain before blocking. In the tables below I use the largest stable or maximum blocking error as a conservative uncertainty estimate.

III Methodology and implementation

A Code structure

The implementation was written as a Python package called `nqs_vmc`. The most important files are listed in Appendix B. The central classes and functions are:

- **GaussianBinaryRBM**: evaluates $\ln \Psi$, wave-function ratios, coordinate derivatives, quantum force, and parameter derivatives.
- **local_energy.py**: evaluates Eq. (18).
- **metropolis.py**: implements brute-force Metropolis sampling.
- **importance_sampling.py**: implements the Langevin Metropolis-Hastings sampler.
- **gradients.py** and **trainer.py**: compute Eq. (23) and optimize the RBM parameters.
- **blocking.py**: computes blocking estimates for correlated samples.

The repository also contains YAML configuration files, selected result files, generated figures, generated tables, and unit tests. This organization makes it possible to reproduce runs without changing the source code.

B VMC algorithm

The following pseudocode summarizes one optimization run.

Listing 1: Pseudocode for the RBM/VMC optimization used in the project.

```
initialize RBM parameters a, b, W
for iteration = 1, ..., n_iterations:
    generate configurations X from |Psi(X)|^2
    for each sampled configuration:
        compute local energy E_L(X)
        compute logarithmic derivatives O_a,
        O_b, O_W
    estimate mean energy <E_L>
    estimate gradients 2(<E_L O> - <E_L><O>)
    update a, b, W using ADAM
    write iteration, energy, error, acceptance
    rate, gradient norm
save final parameters and local-energy samples
```

The same training loop is used for both brute-force and importance sampling; only the sampler changes.

C Tests and validation

The project was developed using tests for the RBM, derivatives, local energy, samplers, blocking routine, and reproducibility. The test suite contained 15 tests and passed before the production runs. The most important validation checks were:

- analytical derivatives were compared with finite-difference derivatives;
- the non-interacting harmonic oscillator reproduced $E = PD/2$;
- the Coulomb potential was tested for the two-particle distance;
- the same random seed reproduced the same result;
- both brute-force and importance samplers returned finite energies and reasonable acceptance rates.

The non-interacting cases are especially important because they test the RBM, local energy, sampler, optimizer, and result-writing pipeline against known analytical values.

D Run parameters

The main non-interacting runs used $\sigma = 1$, small random parameter initialization, ADAM optimization, and fixed random seeds. Brute-force Metropolis used symmetric random-walk proposals, while importance sampling used the time step Δt specified in each configuration. For the interacting system, two longer production runs were analyzed. The first used

$$\begin{aligned} P = 2, \quad D = 2, \quad \omega = 1, \\ N_h = 16, \quad \eta = 0.005, \quad \Delta t = 0.08. \end{aligned} \quad (28)$$

with 30000 samples per optimization iteration and 400 optimization iterations. Motivated by the hyperparameter sweeps, a larger-hidden-layer check used

$$\begin{aligned} P = 2, \quad D = 2, \quad \omega = 1, \\ N_h = 32, \quad \eta = 0.01, \quad \Delta t = 0.08. \end{aligned} \quad (29)$$

with 50000 samples per optimization iteration and 600 optimization iterations. This run was included to test whether the apparently favorable sweep choices combine into a better production calculation. In the interaction potential the code uses `coulomb_epsilon` as a numerical guard: if an inter-particle distance is smaller than this threshold, the local energy is set to infinity. It is not a physical softening of the Coulomb potential; for accepted configurations the interaction is still evaluated as $1/r_{12}$.

IV Results and discussion

A Overall summary

Table 1 summarizes the main completed calculations and is the numerical reference point for the rest of this section. The table makes the contrast between the essentially exact non-interacting tests and the systematically

higher interacting energies clear. The non-interacting harmonic-oscillator benchmarks are reproduced to high precision, validating the analytical local energy, parameter derivatives, samplers, and optimizer. The interacting quantum-dot calculations are less accurate, but the energies remain above the exact value, consistent with the variational principle. The larger-hidden-layer $N_h = 32$ production check did not improve the best energy estimate from the $N_h = 16$ run, illustrating that separately favorable hyperparameters do not necessarily combine into an optimal full calculation. The table separates the naive final-iteration Monte Carlo error from the more conservative blocking error whenever blocking was performed, so that statistical and systematic uncertainties are not conflated.

Table 1: Summary of the main completed energy estimates. Energies, naive errors, blocking errors, and exact values are in atomic units. A dash means that no separate blocking analysis was performed for that row.

Calculation	E	naive err.	block err.	exact
BF, $P = 1, D = 1$	0.499984	$2.1 \cdot 10^{-6}$	–	0.5
BF, $P = 2, D = 2$	1.999999	$3.0 \cdot 10^{-6}$	–	2.0
IS, $P = 1, D = 1$	0.499988	$1.7 \cdot 10^{-6}$	$1.97 \cdot 10^{-5}$	0.5
IS, $P = 2, D = 2$	1.999974	$3.9 \cdot 10^{-6}$	–	2.0
interacting short	3.1476	0.0150	–	3.0
interacting long	3.0826	0.0082	0.0243	3.0
larger N_h check	3.0965	0.0069	0.0201	3.0

B Non-interacting brute-force benchmarks

The first numerical test was the one-particle one-dimensional harmonic oscillator with two hidden nodes. The exact energy is 0.5. Figure 1 shows that the VMC energy converges rapidly to the exact value. The final value was

$$E = 0.499984 \pm 2.1 \cdot 10^{-6}. \quad (30)$$

This confirms that the analytical local energy and RBM parameter optimization are consistent for the simplest case.

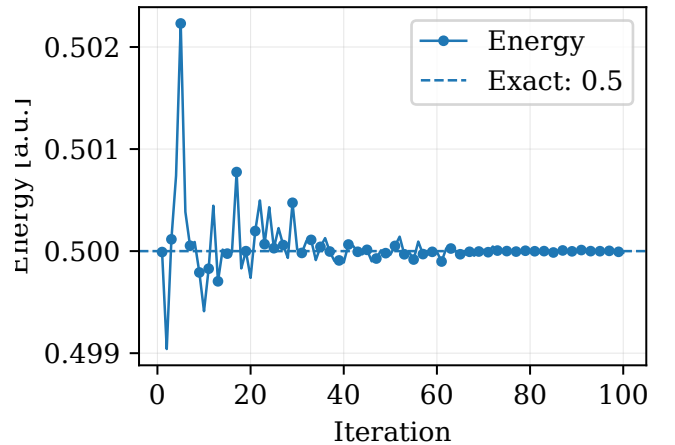


Figure 1: Brute-force Metropolis energy convergence for one particle in one dimension with two hidden nodes. The dashed line is the exact value $E = 0.5$.

The same non-interacting test was repeated for two particles in two dimensions. In this case $P = 2$, $D = 2$, and the exact energy is $E = 2.0$. The result is shown in Figure 2. The final energy was $E = 1.999999 \pm 3.0 \cdot 10^{-6}$. Table 2 gives the final numerical line of the one-dimensional benchmark and is included to document the precision and acceptance rate used for the first validation case.

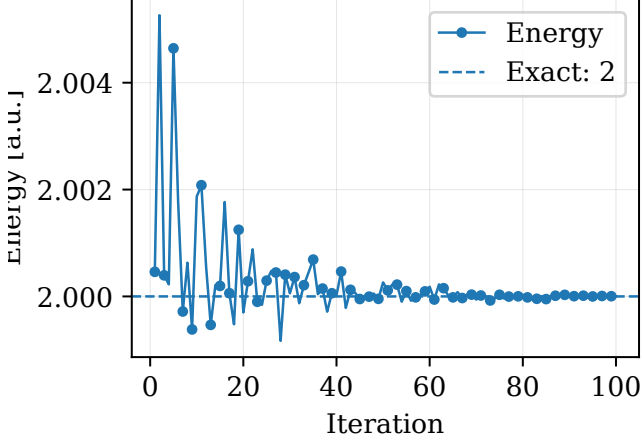


Figure 2: Brute-force Metropolis energy convergence for two particles in two dimensions without interaction. The exact energy is $E = 2.0$.

Table 2: Final iteration of the one-particle one-dimensional brute-force Metropolis benchmark. Energies are in atomic units.

Iter.	E [a.u.]	naive err. [a.u.]	acc.
100	0.499984	$2.1 \cdot 10^{-6}$	0.728

C Hyperparameter sensitivity of the Gaussian benchmark

The learning-rate sweep for the one-dimensional non-interacting problem is shown in Figure 3 and Table 3. Both are needed because the plotted differences are very small on the scale of the benchmark energy. All tested learning rates reproduced the exact energy well, but larger learning rates gave larger deviations and larger statistical errors. The best values in this small sweep were $\eta = 0.005$ and $\eta = 0.03$, both of which gave energies within roughly 10^{-5} of the exact result.

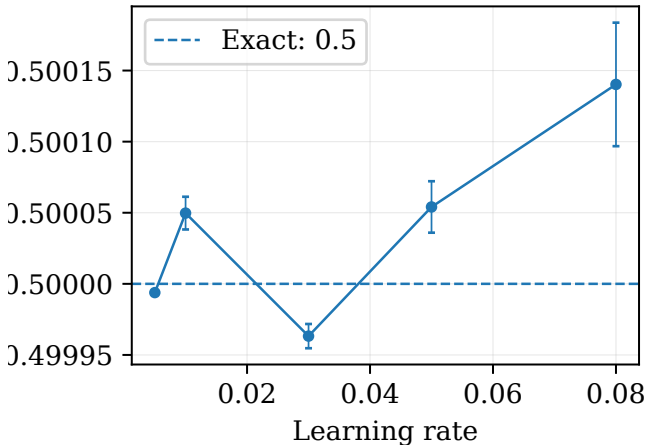


Figure 3: Learning-rate sweep for the one-particle one-dimensional Gaussian benchmark. The scale is very narrow because all points are close to the exact energy.

Table 3: Learning-rate sweep for the one-dimensional non-interacting benchmark. Energies are in atomic units.

η	E [a.u.]	naive err. [a.u.]
0.005	0.499994	$1.6 \cdot 10^{-6}$
0.010	0.500050	$1.2 \cdot 10^{-5}$
0.030	0.499963	$8.5 \cdot 10^{-6}$
0.050	0.500054	$1.8 \cdot 10^{-5}$
0.080	0.500140	$4.3 \cdot 10^{-5}$

The hidden-node sweep is shown in Figure 4 and Table 4. The figure emphasizes the flat dependence on hidden-layer size, while the table gives the small numerical differences. The results show that the simple one-dimensional Gaussian benchmark does not require many hidden nodes. One or two hidden nodes are already sufficient, which is expected because the exact ground state is a Gaussian and the Gaussian part of the RBM can represent it directly.

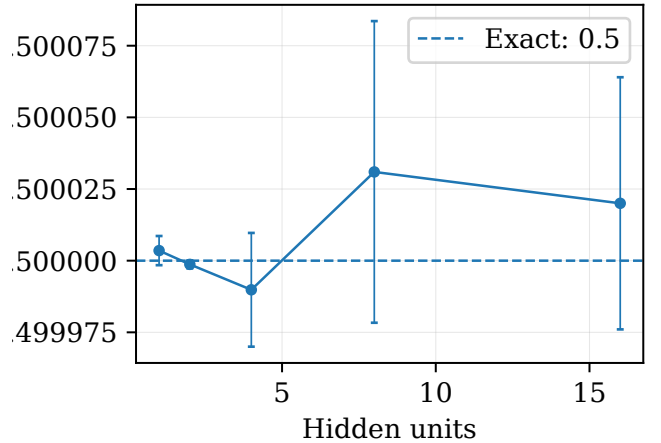


Figure 4: Hidden-node sweep for the one-particle one-dimensional Gaussian benchmark. The plot uses a narrow vertical scale around $E = 0.5$.

Table 4: Hidden-node sweep for the one-dimensional non-interacting benchmark. Energies are in atomic units.

N_h	E [a.u.]	naive err. [a.u.]
1	0.500004	$5.1 \cdot 10^{-6}$
2	0.499999	$1.4 \cdot 10^{-6}$
4	0.499990	$2.0 \cdot 10^{-5}$
8	0.500031	$5.3 \cdot 10^{-5}$
16	0.500020	$4.4 \cdot 10^{-5}$

D Importance sampling benchmarks

Importance sampling was then added using the quantum force from Eq. (25). For the one-particle one-dimensional system, the final energy was

$$E = 0.499988 \pm 1.7 \cdot 10^{-6}. \quad (31)$$

The convergence curve is shown in Figure 5. The acceptance rate was very high, about 0.998, indicating that the time step was conservative.

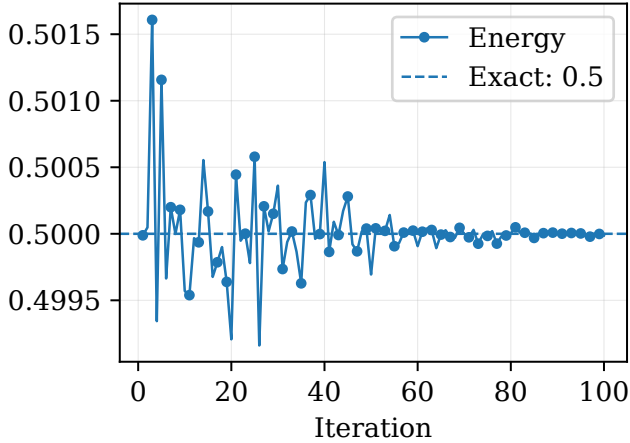


Figure 5: Importance-sampling energy convergence for one particle in one dimension. The exact value is $E = 0.5$.

For the non-interacting two-particle two-dimensional case, the importance-sampling result was

$$E = 1.999974 \pm 3.9 \cdot 10^{-6}. \quad (32)$$

Figure 6 shows that this also converges to the exact value. Compared with brute-force Metropolis, the importance sampler had a much larger acceptance rate, but not necessarily a visibly smaller error in these simple systems. This is because the systems are already easy for brute-force Metropolis. The usefulness of importance sampling becomes more apparent for more complicated systems where random-walk proposals mix slowly.

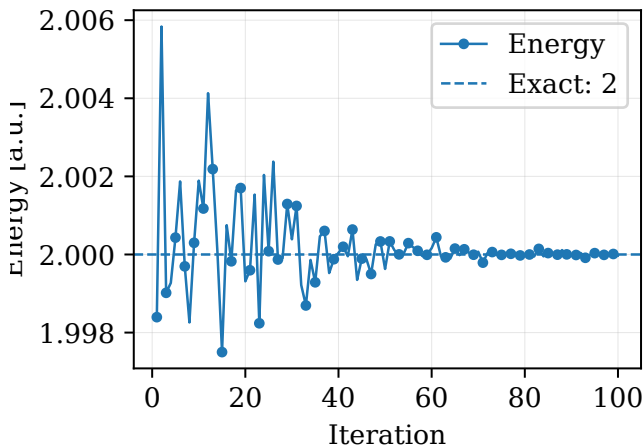


Figure 6: Importance-sampling energy convergence for two particles in two dimensions without interaction. The exact value is $E = 2.0$.

Table 5 gives the direct numerical comparison between brute-force Metropolis and importance sampling for the two non-interacting validation systems. The two methods have comparable final energies, but their sampling behavior is different. Brute-force Metropolis with step size 1.0 gives moderate acceptance rates, especially in the four-coordinate system. Importance sampling with $\Delta t = 0.05$ gives much higher acceptance rates. For these

simple Gaussian systems, however, the high acceptance rate does not lead to a dramatically smaller final error, because the brute-force sampler already performs well. This quantitative comparison supports the use of importance sampling for the interacting study, while also showing that acceptance rate alone is not a sufficient measure of estimator quality.

Table 5: Comparison of brute-force Metropolis (BF) and importance sampling (IS) on the non-interacting benchmarks. All runs used 8000 samples per optimization iteration and 1000 burn-in samples. Energies are in atomic units.

System	sampler	proposal	acc.	E [a.u.]
$P = 1, D = 1$	BF	$s = 1.0$	0.728	0.499984
$P = 1, D = 1$	IS	$\Delta t = 0.05$	0.998	0.499988
$P = 2, D = 2$	BF	$s = 1.0$	0.433	1.999999
$P = 2, D = 2$	IS	$\Delta t = 0.05$	0.993	1.999974

E Autocorrelation and blocking analysis

Blocking was applied to the local-energy samples from the non-interacting importance-sampling run. Figure 7 shows the estimated standard error as the block size increases, while Table 6 lists selected blocking levels used to choose the conservative error estimate. The conservative estimate was

$$\sigma_{\text{block}} = 1.97 \cdot 10^{-5}. \quad (33)$$

This is larger than the naive final-iteration error, as expected for correlated samples. It is therefore a more conservative uncertainty estimate.

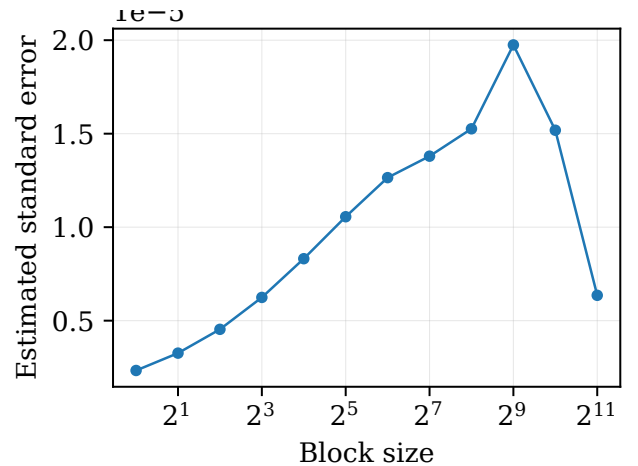


Figure 7: Blocking analysis for the one-dimensional importance-sampling run. The conservative error is taken from the maximum stable block-error estimate.

Table 6: Selected blocking estimates for the non-interacting importance-sampling run.

Level	block size	error [a.u.]
5	32	$1.06 \cdot 10^{-5}$
7	128	$1.38 \cdot 10^{-5}$
9	512	$1.97 \cdot 10^{-5}$
10	1024	$1.52 \cdot 10^{-5}$

F Interacting two-particle quantum dot

The interacting two-particle two-dimensional system is the most demanding test. The exact benchmark is $E = 3.0$ a.u. The first interacting run used $N_h = 8$ and learning rate 0.01. The energy remained around 3.15 at the end of the run, as shown in Figure 8. This run is useful as a baseline, but it is not the best estimate.

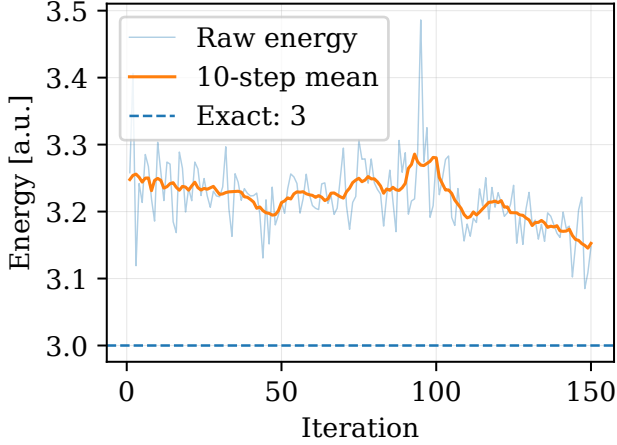


Figure 8: Initial interacting run for two particles in two dimensions. The result remains above the exact value $E = 3.0$ and shows significant fluctuations.

The numerical values from the long interacting production runs are compared in Table 7. A longer run was then performed with $N_h = 16$, learning rate 0.005, time step $\Delta t = 0.08$, 30000 samples per iteration, and 400 optimization iterations. The convergence is shown in Figure 9. The final iteration gave

$$E = 3.08256 \pm 0.00818, \quad (34)$$

while blocking gave a conservative error of 0.0243. The energy is still above the exact value, but the improvement relative to the short run in Figure 8 is clear.

The hyperparameter sweeps suggested that larger hidden layers and a learning rate around 0.01 could improve the result. A larger-hidden-layer production check was therefore performed with $N_h = 32$, $\eta = 0.01$, $\Delta t = 0.08$, 50000 samples per iteration, and 600 optimization iterations. This run gave

$$E = 3.09649 \pm 0.00685, \quad (35)$$

with a conservative blocking error of 0.0201. This did not improve the $N_h = 16$ energy, although it produced a slightly smaller naive error and a comparable blocking error, as seen by comparing Figures 9 and 10 and the values in Table 7, where both error estimates are listed explicitly. The comparison shows that the best settings found in separate one-parameter sweeps do not necessarily combine optimally. It also suggests that the interacting RBM optimization is limited by systematic optimization and ansatz errors, not only by Monte Carlo sampling noise.

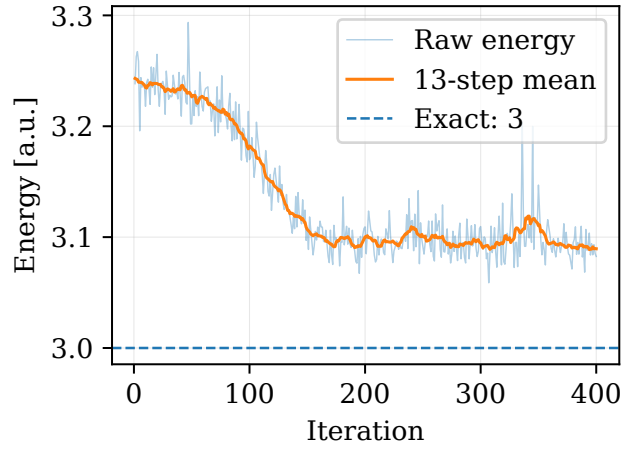


Figure 9: Longer interacting run with $N_h = 16$. The energy decreases from about 3.23 to about 3.08, approaching but not reaching the exact value $E = 3.0$.

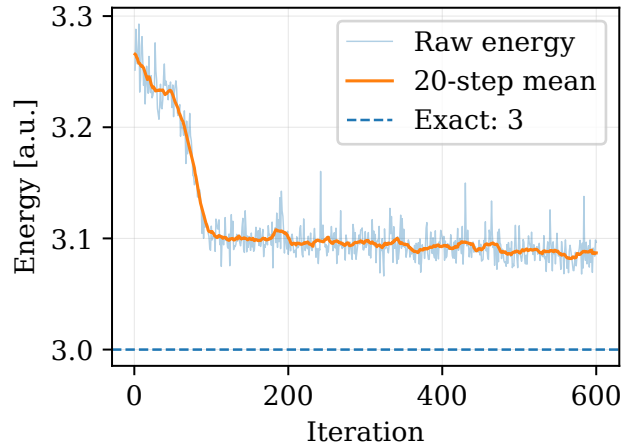


Figure 10: Larger-hidden-layer interacting check with $N_h = 32$, learning rate $\eta = 0.01$, and time step $\Delta t = 0.08$. The final iteration gave $E = 3.09649 \pm 0.00685$, which is higher than the best $N_h = 16$ long run despite using more hidden nodes and more samples.

Table 7: Interacting production runs. The $N_h = 16$ run gave the lowest final energy, while the $N_h = 32$ run is a larger-hidden-layer check. The naive error is the final-iteration Monte Carlo error; the blocking error is the conservative correlated-sample estimate. All energies and errors are in atomic units.

N_h	η	E	naive err.	block err.
16	0.005	3.08256	0.00818	0.0243
32	0.010	3.09649	0.00685	0.0201

The blocking analysis of the two interacting production runs is shown in Figures 11 and 12; these figures justify quoting blocking errors in addition to the naive final-iteration errors.

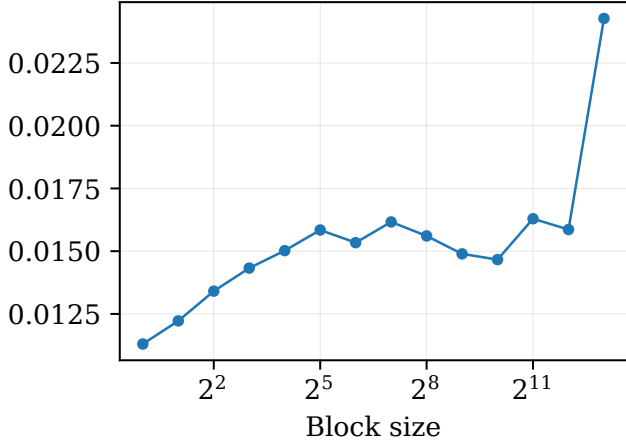


Figure 11: Blocking analysis for the long interacting run. The conservative blocking error is about 0.0243, larger than the naive iteration error.

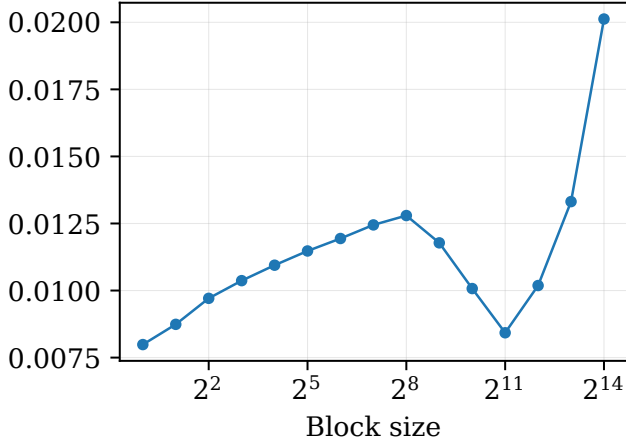


Figure 12: Blocking analysis for the larger-hidden-layer $N_h = 32$ interacting check. The conservative blocking error is about 0.0201, comparable to the long $N_h = 16$ run.

The selected numerical values from the long-run blocking curve are listed in Table 8; the largest value is the conservative error quoted for the $N_h = 16$ result.

Table 8: Selected blocking estimates for the long interacting run with $N_h = 16$.

Level	block size	error [a.u.]
5	32	0.01584
8	256	0.01561
11	2048	0.01629
13	8192	0.02427

G Interacting hyperparameter sweeps

The interacting system was also tested with several hidden-node counts, learning rates, and importance-sampling time steps. These sweeps are not full production runs, but they guide the choice of final parameters.

Figure 13 and Table 9 show the hidden-node sweep. The best value in this sweep was obtained with $N_h = 32$, which gave $E = 3.10254 \pm 0.01461$. The result suggests

that the interacting system benefits from more hidden nodes than the non-interacting system, although the production comparison in Table 7 shows that a larger hidden layer alone is not sufficient to guarantee a better final optimization.

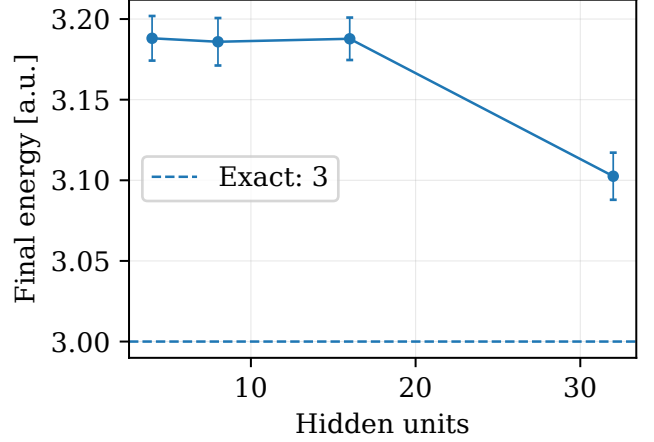


Figure 13: Hidden-node sweep for the interacting two-particle two-dimensional system. The best tested value was $N_h = 32$.

Table 9: Hidden-node sweep for the interacting system.

N_h	E [a.u.]	naive err. [a.u.]
4	3.1881	0.0138
8	3.1859	0.0147
16	3.1878	0.0132
32	3.1025	0.0146

The learning-rate sweep is shown in Figure 14 and Table 10. These results motivated testing a larger learning rate in the final production calculation. The smallest learning rate, 0.002, converged too slowly. The best tested values were 0.01 and 0.02, both giving energies close to 3.08 in the shorter sweep. This motivated the choice of $\eta = 0.01$ for the larger-hidden-layer $N_h = 32$ run, although the combined larger-hidden-layer run did not beat the earlier $N_h = 16$ production calculation.

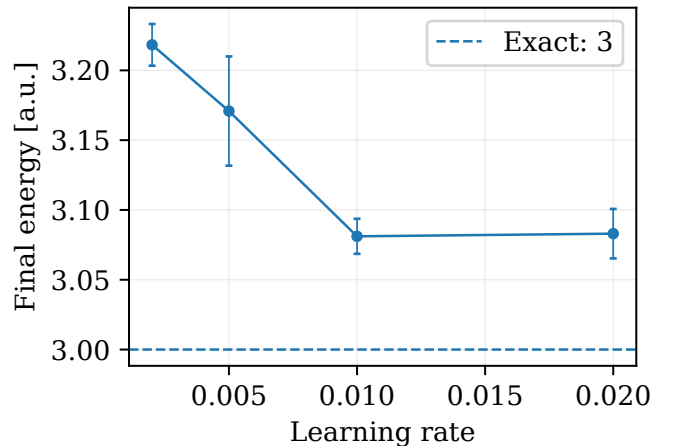


Figure 14: Learning-rate sweep for the interacting system. Learning rates around 0.01 and 0.02 were best among the tested values.

Table 10: Learning-rate sweep for the interacting system.

η	E [a.u.]	naive err. [a.u.]
0.002	3.2183	0.0149
0.005	3.1709	0.0391
0.010	3.0811	0.0126
0.020	3.0830	0.0177

The time-step sweep is shown in Figure 15 and Table 11. These two objects are used together because the figure shows the trend, while the table gives the values needed to choose the production time step. Very small time steps produced high acceptance rates but poor optimization, probably because the chain moved too slowly through configuration space. The best tested time step in this sweep was $\Delta t = 0.08$, although this sweep used learning rate 0.005 and $N_h = 16$, so the conclusion should not be over-interpreted.

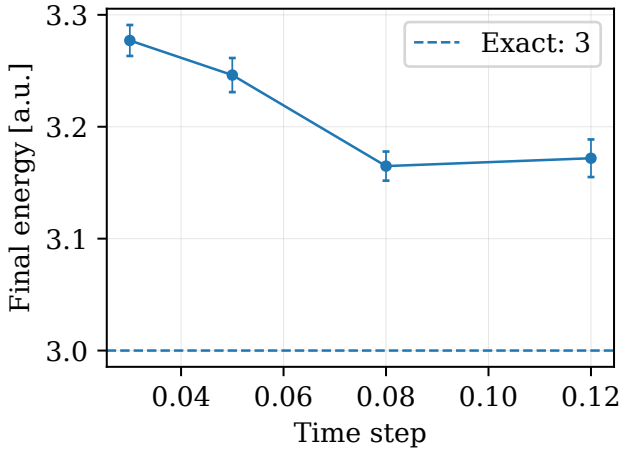


Figure 15: Importance-sampling time-step sweep for the interacting system. The smallest time steps had very high acceptance but worse final energies.

Table 11: Time-step sweep for the interacting system.

Δt	E [a.u.]	naive err. [a.u.]
0.03	3.2771	0.0138
0.05	3.2462	0.0152
0.08	3.1648	0.0130
0.12	3.1719	0.0168

H Reliability and numerical precision

The reliability of the code is strongest for the non-interacting systems, where several independent checks agree with exact analytical results. Both brute-force and importance sampling reproduce the exact energy to roughly 10^{-5} or better. This validates the local-energy expressions, parameter derivatives, samplers, and optimization loop. The sampler comparison in Table 5 also shows that the two independent Markov-chain proposals give consistent energies, which is an additional implementation check.

The interacting system is less precise. The best long run gives an energy about 0.083 above the exact value,

or roughly 2.8% relative error. The larger-hidden-layer $N_h = 32$ check gives an energy about 0.0965 above the exact value, or roughly 3.2% relative error. These deviations are larger than the statistical error estimates, meaning that the dominant error is not sampling noise alone. It is most likely a combination of incomplete optimization, limited expressive power of the present RBM ansatz, and hyperparameter sensitivity. The energies remain above the exact value, which is consistent with the variational principle, but the interacting result is not yet fully optimized.

The blocking curves in Figures 11 and 12, together with Table 8, show that the interacting statistical uncertainty is much larger than in the non-interacting validation case. The blocking samples were taken from the final sampled batch of each optimization run after the burn-in procedure described in Section III. This is sufficient for a conservative project-level error estimate, but a stricter final calculation should freeze the optimized parameters and generate an independent, longer production chain before blocking. The acceptance rates in importance sampling were around 0.99 for many runs. Such high acceptance rates are safe, but they can also indicate that the time step is conservative. The time-step sweep supports this interpretation: the smallest time steps had the highest acceptance rates but did not give the best energies. Future production runs should therefore tune the time step and learning rate together rather than independently and repeat the best settings with several random seeds.

V Conclusion

A Gaussian-binary restricted Boltzmann machine was implemented as a neural quantum state for variational Monte Carlo calculations. The analytical expressions for the local energy, quantum force, and parameter gradients were derived and implemented in closed form. The implementation was validated against non-interacting harmonic oscillator benchmarks, where the exact energy is $E = PD/2$. The final non-interacting estimates were within about 10^{-5} of the exact answers for both brute-force Metropolis and importance sampling. The two-particle electron calculation was interpreted as the symmetric spatial part of a spin-singlet state, so no explicit antisymmetric spatial factor was included.

For the interacting two-particle two-dimensional quantum dot, the exact benchmark is $E = 3.0$ a.u. The best completed long run gave $E = 3.0826 \pm 0.0082$, with a conservative blocking error of 0.0243. A larger-hidden-layer run with $N_h = 32$, $\eta = 0.01$, and more samples gave $E = 3.0965 \pm 0.0069$, with conservative blocking error 0.0201, and therefore did not improve the energy. The interacting calculations are clear improvements over the initial run, but they are still not fully converged to the exact result. Because the difference from the exact value is several times larger than the blocking error, the remaining discrepancy should be interpreted as a systematic optimization or ansatz error, not simply a Monte Carlo error.

The most concrete improvements would be to per-

form a joint sweep over $(N_h, \eta, \Delta t)$ with repeated random seeds, introduce a decaying learning-rate schedule, freeze the optimized parameters and run a separate fixed-parameter production chain for the final blocking analysis, and replace ordinary ADAM optimization by stochastic reconfiguration or a natural-gradient method. A more explicitly correlated trial state, for example by multiplying the RBM by a Jastrow factor or by implementing the optional neural-network/Jastrow ansatz, would also be a natural next step. These changes directly target the dominant limitations observed here: optimization sensitivity and insufficient correlation in the interacting state.

A Analytical details for the RBM ansatz

This appendix gives the derivation used in the code. The wave function is

$$\Psi(\mathbf{X}) = \frac{1}{Z} \exp \left[- \sum_{k=1}^M \frac{(X_k - a_k)^2}{2\sigma^2} \right] \prod_{j=1}^{N_h} (1 + e^{\theta_j}), \quad (36)$$

with

$$\theta_j = b_j + \frac{1}{\sigma^2} \sum_{k=1}^M X_k w_{kj}. \quad (37)$$

Taking the logarithm gives

$$\ln \Psi = \text{const.} - \sum_{k=1}^M \frac{(X_k - a_k)^2}{2\sigma^2} + \sum_{j=1}^{N_h} \ln(1 + e^{\theta_j}). \quad (38)$$

For the first coordinate derivative,

$$\frac{\partial \ln \Psi}{\partial X_k} = - \frac{X_k - a_k}{\sigma^2} + \sum_{j=1}^{N_h} \frac{1}{1 + e^{-\theta_j}} \frac{\partial \theta_j}{\partial X_k} \quad (39)$$

$$= - \frac{X_k - a_k}{\sigma^2} + \frac{1}{\sigma^2} \sum_{j=1}^{N_h} w_{kj} q_j. \quad (40)$$

Using

$$\frac{\partial q_j}{\partial X_k} = q_j(1 - q_j) \frac{w_{kj}}{\sigma^2}, \quad (41)$$

the second logarithmic derivative is

$$\frac{\partial^2 \ln \Psi}{\partial X_k^2} = - \frac{1}{\sigma^2} + \frac{1}{\sigma^4} \sum_{j=1}^{N_h} w_{kj}^2 q_j(1 - q_j). \quad (42)$$

The identity

$$\frac{1}{\Psi} \frac{\partial^2 \Psi}{\partial X_k^2} = \frac{\partial^2 \ln \Psi}{\partial X_k^2} + \left(\frac{\partial \ln \Psi}{\partial X_k} \right)^2 \quad (43)$$

then leads directly to the local energy in Eq. (18).

For the parameter derivatives, the logarithmic derivative with respect to the visible bias is

$$\frac{\partial \ln \Psi}{\partial a_k} = \frac{X_k - a_k}{\sigma^2}. \quad (44)$$

The hidden bias derivative is

$$\frac{\partial \ln \Psi}{\partial b_j} = q_j, \quad (45)$$

and the weight derivative is

$$\frac{\partial \ln \Psi}{\partial w_{kj}} = \frac{X_k q_j}{\sigma^2}. \quad (46)$$

These expressions are inserted into the covariance estimator

$$G_\alpha = 2(\langle E_L O_\alpha \rangle - \langle E_L \rangle \langle O_\alpha \rangle). \quad (47)$$

The quantum force used in importance sampling is

$$F_k = 2 \frac{\partial \ln \Psi}{\partial X_k}. \quad (48)$$

As a consistency check, set $\mathbf{W} = 0$, $\mathbf{a} = 0$, and $\sigma^2 = 1/\omega$. Then

$$\Psi(\mathbf{X}) \propto \exp \left[- \frac{\omega}{2} \sum_{k=1}^M X_k^2 \right], \quad (49)$$

which is the exact product ground state of $M = PD$ independent harmonic-oscillator coordinates. Inserting this in the local energy gives

$$E_L^{(0)} = \frac{M\omega}{2} = \frac{PD\omega}{2}. \quad (50)$$

At $\omega = 1$, this is the benchmark $E = PD/2$ used in the non-interacting calculations.

B Code structure and reproducibility

The code was organized so that the physical model, sampler, optimizer, statistics, and plotting routines are separated from the numerical input parameters. All production runs are controlled by YAML files in `configs/`. This means that the same Python implementation is used for the non-interacting benchmarks, the interacting calculations, and the hyperparameter sweeps; only the configuration file changes between runs.

Table 12 gives the main folders needed to understand and reproduce the project. The detailed file tree in Listing 2 then shows the most important source files. The repository contains additional generated files, but the folders below are the ones that should be inspected first.

Table 12: Main repository folders and their role in reproducibility.

Folder	Role
code/	Source package with the RBM model, Hamiltonian, samplers, optimizer, statistics, I/O, and plotting routines.
configs/	YAML input files. These define the physical system, sampler, optimizer, random seed, and output paths.
scripts/	Command-line programs for experiments, sweeps, blocking analysis, figures, and tables.
tests/	Unit tests and smoke tests used before production runs.
results/	Selected CSV files, processed samples, generated figures, tables, and checkpoints.
report/	Final PDF report, L ^A T _E X source, and Overleaf archive.

Listing 2: Simplified repository tree.

```
FYS4411_Project_2/
  code/nqs_vmc/
    models/gaussian_binary_rbm.py
    physics/local_energy.py
    physics/potentials.py
    sampling/metropolis.py
    sampling/importance_sampling.py
    sampling/quantum_force.py
    optimization/gradients.py
    optimization/optimizers.py
    optimization/trainer.py
    statistics/blocking.py
    io/config_loader.py
    io/result_writer.py
    plotting/
  configs/
  scripts/
  tests/
  results/
  report/
```

The basic reproduction procedure is summarized in Table 13. The setup commands create an isolated environment, install the package, and run the tests. The selected-results command then reruns the main validation cases, interacting runs, parameter sweeps, figure generation, and table generation. I used the same workflow when regenerating the figures and tables for the report.

Table 13: Reproducibility workflow. The exact terminal commands are shown in Listing 3.

Stage	Description
Environment	Create and activate a Python virtual environment.
Installation	Install the Python dependencies and the local <code>nqs_vmc</code> package.
Testing	Run the unit tests for derivatives, local energy, samplers, blocking, and reproducibility.
Selected pipeline	Run the selected experiments and regenerate the report figures and tables.
Manual checks	Inspect selected CSV files, figures, and blocking tables before using them in the report.

Listing 3: Main setup and reproduction commands.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
pip install -e .
pytest -q
python scripts/reproduce_selected_results.py
```

Table 14 lists the configuration groups used for the main numerical results. The individual file names are shown in Listing 4. Keeping these settings in separate YAML files was useful because it made the non-interacting validation runs, the interacting runs, and the parameter sweeps reproducible from the command line without editing the Python source code.

Table 14: Configuration groups used in the calculations.

Group	Purpose
Non-interacting validation	Brute-force and importance-sampling runs for the exact harmonic-oscillator benchmarks.
Blocking analysis	Blocking of saved local-energy samples to estimate conservative statistical errors.
Interacting production	Longer interacting runs for the two-particle two-dimensional quantum dot.
Hyperparameter sweeps	Sweeps over hidden nodes, learning rate, and Langevin time step.
Report generation	Scripts that turn selected CSV files into PDF figures and L ^A T _E X tables.

Listing 4: Selected configuration files.

```
configs/part_b_1p1d_2h_bruteforce.yaml
configs/part_b_2p2d_no_interaction_bruteforce.yaml
configs/part_c_1p1d_2h_importance.yaml
configs/part_c_2p2d_no_interaction_importance.yaml
configs/part_d_blocking_importance.yaml
configs/part_e_2p2d_interacting_long.yaml
configs/part_e_2p2d_interacting_final.yaml
```

```

configs/part_e_blocking_interacting_long.yaml
configs/part_e_blocking_interacting_final.yaml
configs/sweep_part_e_hidden_units.yaml
configs/sweep_part_e_learning_rate.yaml
configs/sweep_part_e_time_step.yaml

```

Listing 5 gives examples of individual commands. These are useful when only one part of the calculation needs to be repeated. For example, after changing the analytical local-energy expression, I first rerun a non-interacting validation case and the tests before using the interacting configurations.

Listing 5: Examples of individual run commands.

```

python scripts/run_experiment.py \
  --config configs/part_b_1p1d_2h_bruteforce.yaml

python scripts/run_experiment.py \
  --config configs/part_c_1p1d_2h_importance.yaml

python scripts/run_experiment.py \
  --config configs/part_e_2p2d_interacting_long.
  yaml

python scripts/run_blocking.py \
  --config configs/part_e_blocking_interacting_long.
  yaml

python scripts/run_sweep.py \
  --config configs/sweep_part_e_hidden_units.yaml

python scripts/make_figures.py
python scripts/make_tables.py

```

The most important reproducibility point is that the benchmark and interacting calculations share the same implementation. The non-interacting harmonic-oscillator runs therefore test the same local-energy, sampling, optimization, and I/O code that is later used for the Coulomb-interacting system. This makes the exact non-interacting results a direct check of the numerical pipeline rather than a separate demonstration program.

C Additional comments on limitations

The present implementation uses closed-form derivatives and ADAM optimization. This was sufficient for the non-interacting benchmarks, but the interacting problem indicates that more advanced optimization may be useful. Stochastic reconfiguration or natural-gradient methods are often more stable for variational wave functions because they account for the geometry of the parameterized state space. A second limitation is that each hyperparameter sweep used relatively short runs and only one random seed. The sweep results should therefore be interpreted as guidance rather than final high-precision estimates.

A further limitation is the production protocol for uncertainty estimation. The code saves local-energy samples from the last optimization iteration for blocking. A more rigorous protocol would save the optimized parameters, restart the sampler from several independent initial configurations, discard burn-in for each chain, and

then block the fixed-parameter production samples. This would separate optimization noise from sampling noise more cleanly.

The optional neural-network replacement of the RBM was not performed. A placeholder file for a neural-Jastrow model exists in the repository, but the reported results all use the Gaussian-binary RBM ansatz. Finally, the implementation does not include Slater determinants or explicit antisymmetrization, so it should not be used for general fermionic excited states without modification.

References

- [1] G. Carleo and M. Troyer, “Solving the quantum many-body problem with artificial neural networks,” *Science* **355**, 602–606 (2017).
- [2] M. Taut, “Two electrons in an external oscillator potential: Particular analytic solutions of a Coulomb correlation problem,” *Physical Review A* **48**, 3561–3566 (1993).
- [3] G. Torlai and R. G. Melko, “Machine-learning quantum states in the NISQ era,” *Annual Review of Condensed Matter Physics* **11**, 325–344 (2020).
- [4] G. E. Hinton, “A practical guide to training restricted Boltzmann machines,” Technical Report UTML TR 2010-003, University of Toronto (2010).
- [5] H. Saito, “Method to solve quantum few-body problems with artificial neural networks,” *Journal of the Physical Society of Japan* **87**, 074002 (2018).
- [6] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” International Conference on Learning Representations (2015).
- [7] H. Flyvbjerg and H. G. Petersen, “Error estimates on averages of correlated data,” *Journal of Chemical Physics* **91**, 461–466 (1989).
- [8] Computational Physics II FYS4411/FYS9411, “Project 2: Machine learning for quantum many-body problems,” University of Oslo, Spring 2026. <https://github.com/CompPhysics/ComputationalPhysics2/tree/gh-pages/doc/Projects/2026/Project2/Project2ML>
- [9] Computational Physics II FYS4411/FYS9411, “Lecture notes on Boltzmann machines,” University of Oslo. https://compphysics.github.io/ComputationalPhysics2/doc/LectureNotes/_build/html/boltzmannmachines.html